ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

On Combiners for Verifiable Random Functions

Bachelor Thesis

Basil Rohner

March 17, 2026

Advisors: Prof. Dr. A. D. Visor, Dr. P. Ostdoc
Department of Computer Science, ETH Zürich

Acknowledgments

Abstract

Contents

Contents **iii**

1 Introduction **1**
1.1 Motivation and Previous Work 1
1.2 Technical Overview 4

2 Notation and Preliminaries **6**
2.1 Notation 6
2.2 Verifiable Functions 7
2.3 Robust Combiners 11

3 Results **13**
3.1 Robust Black-Box DAG Combiner 13
3.2 Impossibility of Robust Black-Box VRF-Combiner 16
3.3 Discussion and Extensions 22

4 Conclusion **23**

Appendix **24**
Proof of **??** 24

Chapter 1

Introduction

1.1 Motivation and Previous Work

Verifiable Random Functions. Verifiable random functions (VRF) [?] are cryptographic primitive that allows the owner of a secret key to obtain output-proof pairs by executing an evaluation algorithm on specified points such that the outputs pass all computational tests for randomness with high probability in the absence of the corresponding proof, and such that the correct evaluation of any image can be verified by a party with access to the corresponding public key and proof. In practice VRFs are used in signatures with strong verifiability guarantees [?], resettable zero-knowledge proofs [?], lottery systems [?, ?], transaction escrow schemes [?], updatable zero-knowledge databases [?], e-cash systems [?], and in Proof-of-Stake blockains through leader election protocols [?, ?, ?, ?]. In addition to the standard properties, VRFs have been studied with regards to additional, more advanced, properties such as public and secret-key aggregability [?] in the context of efficiently verifiable blockchain protocols [?] and unbiasability[?] in the context of distributed leader election protocols[?] and additional fairness guarantess in DeFI [?].

additional applications

We also consider primitives with relaxed properties such as verifiable unpredictable functions (VUFs) [?], which only require the outputs of the evaluation function to be unpredictable instead of pseudorandom, and weak verifiable random functions (wVRFs) [?] that only require pseudorandomness on the image of random points. While VRFs can be reduced to VUF using a hardcore bit construction [?], there exists no black-box reduction from VRF to wVRF which the authors [?] show using a separation using enhanced trapdoor permutations.

Aggregable Verifiable Random Functions. We consider aggregability of VRFs with respect to their secret-keys, also referred to as key-homomorphism,

and with respect to their public-key. Key-homomorphism requires the function underlying the evaluation algorithm to constitute a group homomorphism form a group structure on the secret-key space to a group structure on the output codomain for any fixed evaluation point . On the other hand a VRF is public-key aggregable if there exists an algorithm for aggregating image-proof pairs into a single image-proof pair and an algorithm that given an aggregated image-proof pair and the corresponding set of verification keys verifies the aggregated image with respect to a fixed evaluation point. A seicure aggregable VRF is a public-key aggregable VRF that satisfies .

same must hold for verification algorithm

properties

Unbiasable Verifiable Random Functions. VRFs are used in Proof of Stack (PoS) blockchain systems for secret leader election protocols [**?**]. A leader election protocol is a distributed algorithm in which multiple parties determine a leader in a way that the leader is kept secret and such that no subset of colluding malicious parties may influence the result of the election. Generally this is done by having every party evaluate the VRF at a given point and selecting the party with the smallest result as the leader. Original VRF don't provide strong enough security guarantees such that the described protocol is secure however since even for a secure VRF there might exist key pairs for which the outputs of the VRF could be biased towards lower values. Unbiasability [**?**] is a security property that ensures that even if the adversary may arbitrarily choose the keys, the output distribution of the evalution cannot be skewed.

Robust (Black-Box) Combiners. Robust combiners are constructions that take many instances of the same cryptographic primitive, called candidate schemes, in form of a description of their Turing machine, and securely implement that cryptographic primitive if a threshhold of the candidate schemes are secure. Since we generally intend for the combiner to derive its security from the subset of secure candidate schemes, and not by implementing a secure instance of the underlying primitive itself and ignoring the candidates, we require black-box reductions from the combiner construction to any sufficiently large subset of its candidates. In this work mainly consider robust black-box combiners, a subclass of combiners that restricts access of the combiner to its candidate schemes to oracles which can be queried but not inspected for their underlying description. Robust combiners allow the hedging of trust in cryptographic hardness assumptions underlying computational security properties, by basing their security on a subset of hard problems rather than a a single hard problem. In practice robust combiners are uesful to reduce the dependency of a systems on the security of a specific third party implementation, e.g. a cryptographic library. In theory robust combiners are related to universal cryptographic construction, i.e. constructions that implement a secure instance of a cryptographic primitive under

the only assumption that a secure instance of the cryptographic primitive exists, a prominent example is Levin's universal one-way function[**?**]. Finally combiners play a central role in the transition towards post-quantum cryptography where hardness assumptions are not as well studied as the respective assumptions for the classic cryptography counterpart, in particular the combination of classical and post-quantum cryptographic assumptions has been a fruitful research field.

Combiners have been considered for a wide variety of cryptographic primitivey, including key exchange protocols [**?**], signature schemes [**?**], oblivious-transfer [**?**], zero-knowledge proofs [**?**], commitment schemes [**?**], and indistinguishability obfuscation [**?**]. While pseudorandom functions (PRFs) admit easy robust $(n;1)$ combiners by simply XOR'ing the outputs of the candidate schemes, evaluated on independent keys the combination of VRFs, the combination of VRFs is less obvious. To the best of our knowledge, there has been no prior research on robust combiners for verifiable random functions.

Our Contribution

We introduce a new definition of a robust black-box combiner, a robust black-box DAG combiner, which models the implemented algorithms of the combiner as computations that follows the structure of a vertex-colored directed acyclic graph starting from a source and ending in a sink, where for each vertex in the graph we query a specific oracle of a specific candidate on the output of a probabilistic polynomial time Turing machine of all predecessors. We show that the notion of a robust black-box DAG combiner is sufficiently strong to capture all ppt. robust black-box combiners. We use this new model to show the impossibility of combining multiple instances of wVRFs which extends to the impossibility of combining VUFs and VRFs without additional assumptions. We argue that the combination of the unique provability property is however less important since it is of information theoretic or statistical nature and therefore doesn't rely on any computational hardness assumption. In fact, good VRF constructions such as ECVRF [**?**], lattice VRF [**?**] and [**?**] mathematically prove unique provability mathematically, rendering combination less important. We consider the possibility to combine more advanced security properties on VRFs such as unbiasability for which we construct a combiner for uniform constructions which relies on collision resistant hash functions and randomness extractors to combine unbiasable VRFs under the additional assumption that all candidate schemnes are uniquely provable. We also investigate combiners for secret-key aggregability and public-key aggregability. Secret-key aggregability is a publicly verifiable property due to an efficient algorithm by Blum, Luby and Rubinfeld [**?**] which allows checks for the homomorphism

property of a function in time independent of the size of the underlying groups. Combiners for publiy-key aggregability Finally we investigate additional properties such as constrainability [**?**] nd which all admit the simple pseudorandomness combiner construction.

1.2 Technical Overview

Robust Black-Box DAG Combiner. We present another specification of robust black-box combiners, termed robust black-box DAG combiners, which describe a form of black-box combiners that are structured by a directed acyclic graph for each implemented algorithm of the underlying cryptographic primitive in Definition 3.1. We show that the newly introduced notion in Definition 3.1 is in fact equivalent to the standard definition of robust black-box combiners Definition 2.16, a result we formalize in the following theorem.

Theorem 1.1

Impossibility of Black-Box VRF Combiner. Subsequently we show that there exists no robust $(n;k)$ black-box combiner that takes as input n candidates, of which k securely implement a VRF according to **??** the combiner implements a VF in **??** that is statistically correct according to Definition 2.4, uniquely provable according to Definition 2.5, and weakly pseudorandom according to Definition 2.7. That is even assuming that $n-1$ candidates implement secure VRFs according to **??** the combiner cannot guarantee to implement a statistically correct weak VRF. We formally state this in the following theorem.

Theorem 1.2

Relaxed Combiners for VRF. Having established Theorem 3.3 we move on by assuming that every candidate scheme is uniquely provable, which enables a simple robust $(n;1)$ black-box combiner for VRF in Figure 3.4 which samples independent keys per candidate generically for the key-generation algorithm, for evaluation the combiner evaluates every candidate on the input with their corresponding secret key, returns the XOR of the images with the proof contiaining all views of the candidate schemes. For verification the combiner verifies that all candidates views in the proof are consistent by querying every candidates verification oracle and then recomputes the output to compare it with the image. The construction allows us to state the following theorem.

Theorem 1.3

Relaxed Combiners for Unbiasable VRF. Having relaxed the unique provability property of the candidates we consider combiner constructions for more advanced notions of VRFs. A VRF is fully unbiasable if no efficient adversary can craft a set of keys such that the output distribution distribution is skewed into any direction. We show that fully unbiasable VRF as defined in **??** admit non-uniform robust $(2;1)$ black-box combiners that base their property on collision resistant hash functions and 2-source randomness extractors. . Formally we formulate the following theorem.

this is unclear

restate

Theorem 1.4

Relaxed Combiners for Aggregable VRF. Then we consider aggregable VRF we define in **??** which additionally allow the aggregation of many image-proof pairs into a single image-proof pair to realize efficient batch verification. Key-homomorphic, secret-key aggregable, VRF require that the evaluation algorithm forms a group homomorphism with respect to group structures on the secret-key space and the product of the output codomain and the proof space. Using efficient grouphomomorphism tests by [**?**] we can construct a $(n;1)$ robust combiner for statistically key-homomorphic VRF that works by publicly checking the homomorphism property before running the algorithm.

restate

Theorem 1.5

For public-key aggregability

Relaxed Combiners for Constrainable VRF. Finally we consider constrainable VRF, defined in **??**. Constrainable VRF additionally allow the user to generate constrained keys, which only allow the evaluation on a constrained set of evaluation points.

Chapter 2

Notation and Preliminaries

2.1 Notation

For a positive integer n we define $[n] = \{1, \ldots, n\}$. Unless specified otherwise we use λ to denote the security parameter. A function $\varepsilon(n)$ is called negligible, if it vanishes faster than the inverse of any polynomial in n. We use $\mathsf{poly}(n)$ to denote the set of functions polynomial in n and $\mathsf{negl}(n)$ to denote the set of functions negligible in n. For a set S we write $y \leftarrow_{\$} S$ to indicate that the value of y is sampled uniformly at random from S, for a random variable X we write $y \leftarrow_{\$} X$ to indicate that the value of y is sampled from the corresponding sample space with the underlying distribution, and for a randomized algorithm $\mathcal{A}$ we write $y \leftarrow_{\$} \mathcal{A}(x)$ to assign to y the value of the output of $\mathcal{A}$ on x. We use $\leftarrow$ for simple assignment. We denote column vectors with lower-case bold letters (e.g. $\mathbf{x}$) . Unless specified otherwise we assume that any algorithm gets as input in the first argument the unary encoding of the security parameter 1^{λ}. Finally we use ppt. to abbreviate probabilistic polynomial time in context of turing machines and algorithms.

I needed this previously bu

Definition 2.1 (Polynomial Functions) *We say that a function $f : \mathbb{N} \to \mathbb{N}$ is polynomial in its argument λ if*

$$\exists c, d \in \mathbb{N}, \forall \lambda \in \mathbb{N}, f(\lambda) \leq c\lambda^d$$

Definition 2.2 (Negligible Functions) *We say that a function $f : \mathbb{N} \to \mathbb{R}$ is negligible in its argument λ if*

$$\forall c \in \mathbb{N}^+, \exists n_0 \in \mathbb{N}, \forall \lambda > n_0, |f(\lambda)| < \frac{1}{x^c}$$

2.2 Verifiable Functions

A verifiable function (VF) VF is a cryptographic primitive that implements a key-generation algorithm $\mathsf{Gen}_{\mathsf{VF}}$ to sample a key-pair $(\mathsf{sk}, \mathsf{pk})$ containing a secret and a public key, a deterministic evaluation algorithm $\mathsf{Eval}_{\mathsf{VF}}$ that allows the holder of a secret key sk to make evaluations at any points $x \in \{0,1\}^\lambda$ to obtain an image $y \in \mathcal{Y}$ and a corresponding proof $\pi \in \mathcal{P}$, and a verification algorithm that allows a different party with access to the corresponding public key pk to verify that y was correctly computed from x if given access to the proof π. The definition takes inspiration of the definition of a verifiable random function (VRF) as defined by [**?**], to reduce redundand when considering VRFs with additional or relaxed properties we define the properties seperately on top of a verifiable function (VF).

Definition 2.3 (Verifiable Function) *A verifiable function (VF)* VF *is a triple of efficient (ppt.) algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *with the syntax*

this is less elegant now

- $\mathsf{Gen}_{\mathsf{VF}}$*: The key-generation algorithm takes as input the unary encoding of the security parameter and produces a pair* $(\mathsf{sk}, \mathsf{pk}) \in \mathcal{SK} \times \mathcal{PK}$ *containing a secret and a public key.*
- $\mathsf{Eval}_{\mathsf{VF}}$*: The evaluation algorithm takes as input a tuple* (sk, x) *containing a secret key* $\mathsf{sk} \in \mathcal{SK}$ *and an evaluation point* $x \in \{0,1\}^\lambda$ *and produces an tuple* (y, π) *containing an image* $y \in \mathcal{Y}$ *and a corresponding proof* $\pi \in \mathcal{P}$*.*
- $\mathsf{Vfy}_{\mathsf{VF}}$*: The verification algorithm takes as input a tuple* (pk, x, y, π) *containing a public key* $\mathsf{pk} \in \mathcal{PK}$*, an evaluation point* $x \in \{0,1\}^\lambda$*, an image* $y \in \mathcal{Y}$ *and a proof* $\pi \in \mathcal{P}$ *and returns a bit* b *denoting acceptance or rejection.*

A VF can assume a range of different attributes. Based on the satisfied properties we arrive at specific constructions such as verifiable random functions (VRFs) or verifiable unpredictable functions (VUFs). We first describe the correctness property of a VF. Informally a VF is correct if for any key-pair produced honestly from the key-generation algorithm and any point $x \in \{0,1\}$ the output (y, π) of the evaluation alogrithm is accepted by the verification algorithm.

Definition 2.4 (ε-Correctness) *A verifiable function* $\mathsf{VF} = (\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is ε-correct, if for all security parameters $\lambda \in \mathbb{N}^+$ we have that*

$$\Pr\left[\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = 1 : \begin{array}{r}(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda) \\ x \leftarrow_{\$} \{0,1\}^\lambda \\ (y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)\end{array}\right] \geq \varepsilon(\lambda)$$

An ε-correct VF is defined to be perfectly correct, or just correct, if $\varepsilon = 1$, and it is defined to be statistically correct if $\varepsilon = 1 - \mu$ for a negligible function μ.

While correctness requires all honestly generated images to be accepted by the verification algorithm we additionally demand that the verification algorithm rejects all maliciously generated images. We define the property of unique provability to be the property that requires that for every evaluation point $x \in \{0,1\}^\lambda$, and every public-key $\mathsf{pk} \in \mathcal{PK}$ only one image $y \in \mathcal{Y}$ is accepted by the verification algorithm.

Definition 2.5 (Unique Provability) *A verifiable function* VF*, with algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$*, defined to be uniquely provable if for all evaluation points* $x \in \{0,1\}^\lambda$*, a public-keys* $\mathsf{pk} \in \mathcal{PK}$ *(even maliciously crafted ones) and any two images-proof pairs* $(y_0, \pi_0), (y_1, \pi_1) \in \mathcal{Y} \times \mathcal{PK}$*:*

$$\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_0, \pi_0) = 1 \land \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_0, \pi_0) = 1 \implies y_0 = y_1$$

We define a secure VF to be a VF that satisfies both correctness and unique provability. We define the computational security properties of pseudorandomness, weak pseudorandomness and unpredictability.

maybe separate correct and secure here or maybe omit it fully

Definition 2.6 ((t,ε)-Pseudorandomness) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* (t,ε)*-pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the prf security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is pseudorandom, if it is (t,ε)*-pseudorandom with* ε *being a negligible function and* t *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$*.*

Definition 2.7 ((t,ε)-weakly Pseudorandomness) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* (t,ε)*-weakly pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the wprf security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is weakly pseudorandom, if it is (t,ε)*-weakly pseudorandom with* ε *being a negligible function and* t *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$*.*

Definition 2.8 ((ε,t)-Unpredictability) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* (t,ε)*-unpredictable if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the up security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

Experiment $\mathbf{Exp}^{\text{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

$b \leftarrow_{\$} \{0,1\}$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda)$
$S \leftarrow \varnothing$
$(x^*, \sigma) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^\lambda, \mathsf{pk})$
$(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
$y_1 \leftarrow_{\$} \mathcal{Y}$
$b' \leftarrow_{\$} \mathcal{A}(1^\lambda, \sigma, y_b)$
return $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

$b \leftarrow_{\$} \{0,1\}$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda)$
$S \leftarrow \varnothing$
$x^* \leftarrow_{\$} \{0,1\}^\lambda$
$(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
$y_1 \leftarrow_{\$} \mathcal{Y}$
$b' \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^\lambda, \mathsf{pk}, x^*, y_b)$
return $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

$(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^\lambda)$
$S \leftarrow \varnothing$
$(x^*, y^*) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^\lambda, \mathsf{pk})$
$(y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
return $y^* = y \wedge x^* \notin S$

Oracle $\mathcal{O}_{\text{eval}}(x)$

$S \leftarrow S \cup \{x\}$
$(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
return y

Figure 2.1: Security game for pseudorandomness, weak pseudorandomness and unpredictability.

$$\mathbf{Adv}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is unpredictable, if it is $(\mathfrak{t}, \varepsilon)$*-unpredictable with* ε *being a negligible function and* $\mathfrak{t}$ *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), \mathfrak{t} \in \mathsf{poly}(\lambda)$.

We define a verifiable random function (VRF) to be a secure pseudorandom VF, a verifiable unpredictable function (VUF) to be a secure unpredictable VF and a weaker verifiable random function (wVRF) to be a secure weakly pseudorandom VF. We formally define the primitives VRF, VUF and wVRF in the following definitions.

Definition 2.9 (Verifiable Random Functions) *A verifiable random function* $(\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ *is a VF that satisfies perfect correctness, unique provability and pseudorandomness.*

Definition 2.10 (Verifiable Unpredictable Functions) *A verifiable random function* $(\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ *is a verifiable function that satisfies perfect correctness, unique provability and unpredictability.*

Definition 2.11 (Weak Verifiable Random Functions) *A verifiable random function* $(\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ *is a verifiable function that satisfies perfect correctness, unique provability and weak pseudorandomness.*

Unbiasability, following the definition of **[?]**, is a property for VRFs that prevents adversaries from skewing the output distribution of the evaluation function on malleable keys. To formally definine unbiasability we require the notion of monotone predicates.

Definition 2.12 (Monotone Predicates [?]) *A probabilistic predicate* ρ *on the domain* $(\mathcal{Y} \cup \{\perp\})^*$ *is monotone, iff for all* $y \in \mathcal{Y}^*$*, all index sets* $I \subseteq [|y|]$ *and every bitstring of randomness* $r \in \{0,1\}^\lambda$ *we have*

$$\Pr[\rho(\tilde{y}; r) = 1] \leq \Pr[\rho(y; r) = 1]$$

where $\tilde{y}_i = y_i$ *if* $i \notin I$ *and* $\tilde{y}_i = \perp$ *otherwise.*

Definition 2.13 (Unbiasability [?]) *A VRF* VRF *is (fully)* $(\mathfrak{t}, \varepsilon, \mathfrak{a}, \mathfrak{b})$*-unbiasable if for every stateful adversary* $\mathcal{A}$ *running in time at most* $\mathfrak{t}$ *its advantage in the unbiasability security experiment in* **??** *is bounded by*

$$\mathbf{Adv}^{\mathrm{fub}}_{\mathsf{VRF}}(\lambda, \mathcal{A}) := 2 \cdot \Pr\left[\mathbf{Exp}^{\mathrm{fub}}_{\mathsf{VRF}}(\lambda, \mathfrak{a}, \mathfrak{b}, \mathcal{A}) = 1\right] - 1 \leq \varepsilon(\lambda)$$

We say that a VRF is unbiasable if it is $(\mathfrak{t}, \varepsilon, \mathfrak{a}, \mathfrak{b})$*-unbiasable and* ε *is a negligible function in the security parameter* λ*, i.e.* $\varepsilon \in \mathsf{negl}(\lambda)$.

Key-homomorphism of VF requires that the secret-key space and the product space of the image and proof space admit group structures $(\mathcal{SK}, \odot)$ and $(\mathcal{Y} \times \mathcal{P}, \otimes)$ respectively. Intuitively a VF is key-homomorphic if for every evaluation point $x \in \{0,1\}^\lambda$ the evaluation algorithm on x implements a grouphomomorphism in the secret key.

Definition 2.14 (Key-Homomorphism) *A VF* VF *is key-homomorphic if for every evaluation point* $x \in \{0,1\}^\lambda$*, secret key* $\mathsf{sk}_1, \mathsf{sk}_2 \in \mathcal{SK}$ *we have*

$$\mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}_1 \odot \mathsf{sk}_2, x) = \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}_1, x) \otimes \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}_2, x)$$

We also refer to key-homomorphism as secret-key aggregability. To define public-key aggregability we first define the cryptoraphic primitive of an aggregable verifiable function (AggVF) that additionally implements algorithms to combine image-proof pairs into a single image-proof pair aggregate.

think I should abandon the subsetting of the primitive and introduce . notation. Also use subexp **P** for primitive in combiner

Experiment $\mathbf{Exp}^{\mathrm{fub}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

$b \leftarrow_{\$} \{0,1\}$
$(\rho, \mathsf{pk}_1, \ldots, \mathsf{pk}_{\mathfrak{a}(\lambda)}) \leftarrow_{\$} \mathcal{A}(1^\lambda)$
require ρ is monotone
$x_1, \ldots, x_{\mathfrak{b}(\lambda)} \leftarrow_{\$} \{0,1\}^\lambda$
$(y^0_{1,1}, \pi^0_{1,1}), \ldots, (y^0_{\mathfrak{a}(\lambda),\mathfrak{b}(\lambda)}, \pi^0_{\mathfrak{a}(\lambda),\mathfrak{b}(\lambda)}) \leftarrow_{\$} \mathcal{A}(x_1, \ldots, x_{\mathfrak{b}(\lambda)})$
for $i \in [\mathfrak{a}(\lambda)], j \in [\mathfrak{b}(\lambda)]$ **do**
  $\beta_{i,j} \leftarrow \mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_i, x_j, y^0_{i,j}, \pi^0_{i,j})$
  if $\beta_{i,j} = 1$ **then**
    $z_{i,j} \leftarrow y^b_{i,j}$
  else $z_{i,j} \leftarrow \perp$
$r \leftarrow \{0,1\}^\lambda$
return $\rho(z_{1,1}, \ldots, z_{\mathfrak{a}(\lambda),\mathfrak{b}(\lambda)}, r) \neq b$

Figure 2.2: Full unbiasability security experiment.

2.3 Robust Combiners

Given a cryptographic primitive P we define a robust $(n;k)$ P-combiner

Definition 2.15 (Robust $(n;k)$ P-combiner) *Given a cryptographic primitive* P *describing p algorithms* $(\mathsf{alg}_\mathsf{P}{}^1, \ldots, \mathsf{alg}_\mathsf{P}{}^2)$*, a robust* $(n;k)$ P*-combiner is a p-tuple of algorithms that take as input the description of n Turing machines* $\mathcal{O}_1, \ldots, \mathcal{O}_n$*, the candidate schemes*

$$\Pi_\mathbf{P}(\mathcal{O}_1, \ldots, \mathcal{O}_n) = \left(\Pi_\mathbf{P}(\mathcal{O}_1, \ldots, \mathcal{O}_n).\mathsf{alg}_\mathsf{P}{}^1, \ldots, \Pi_\mathbf{P}(\mathcal{O}_1, \ldots, \mathcal{O}_n).\mathsf{alg}_\mathsf{P}{}^n\right)$$

that run in time polynomial in n, and the size of their input, such that the combiner securely implements the primitive P *if at least k candidates schemes securely implement the primitive* P*.*

We write robust combiner for robust $(n;k)$ P-combiner whenever n, k and P are clear from the context. A robust combiner gets the candidate schemes as a description of their turing machine and can therefore be adaptive in the code of its candidates. Note that this definition allows the robust combiner to ignores its candidate schemes and implement a secure instance of the underlying primitive indepdently of its candidates. One can therefore always construct a robust combiner for primitives that admit secure instances, by contraposition this also means that any result that rules out the existence of a robust combiner for a specific primitive P also rules out the existence

of the primitive itself. Intuitively we would like for the combiner to derive its security from the security of its secure candidate schemes, therefore we define robust black-box combiner. A robust black-box combiner is a robust combiner that, instead of getting access to the description of the Turing machines of the candidate schemes, only gets oracle access to the underlying candidate schemes. We define the notion of a robust black-box combiner closely following the definition by [**?**]. Note that we explicitly require a robust black-box combiner to derive its security from its candidates, for that reason we require the security to hold relative to a PSPACE oracle PSPACE.

Very unhappy with this definition

Definition 2.16 ($(n;k)$-Robust Black-Box P-Combiner) *Given a cryptographic primitive* P *describing p algorithm* $(\mathsf{alg_P}^1, \ldots, \mathsf{alg_P}^p)$*, a robust* $(n;k)$ *black-box* P*-combiner is a p-tuple of oracle Turing machines with access to n oracles, the candidate schemes* $\mathcal{O}_1, \ldots, \mathcal{O}_n$

$$\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n} = \left(\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{alg_P}^1, \ldots, \Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{alg_P}^n\right)$$

A $(n;k)$*-robust combiner* $\Pi_{\mathbf{P}}$ *for primitive P is defined to be black-box, denoted by* Π_{P}*, if additionally it satisfies that if there exists an adversary* $\mathcal{A}$ *(possibly computationally unbounded) that breaks the security of* $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$*, then there exist at black-box reductions with oracle access to* $\mathcal{A}$ *for at least* $n - k + 1$ *candidates.*

We remark that compositite security definitions that rely on multiple properties are broken if there exists an adversary that breaks a security property with significant advantage. Note that if the security would not hold relative to a PSPACE oracle PSPACE then we would find that a secure implementation of the primitive would indeed implement a secure robust black-box combiner since there exists no adversary breaking the security properties of P.

Chapter 3

Results

3.1 Robust Black-Box DAG Combiner

Intuitively, we imagine a robust black-box combiner to operate by orchestrating a sequence of queries to its candidate schemes with computations that we model with probabilistic turing machines in between. To make the combiner structure more explicit we will model it with a vertex-colored directed acyclic graph (DAG) where the nodes correspond to an oracle call to a candidate scheme specified by the vertex-coloring with inputs computed from the outputs of the oracle calls of the predecessors using a ppt algorithm. Formally we introduce the notion of a robust black-box DAG combiner:

Definition 3.1 (Robust $(n;k)$ Black-Box DAG P-Combiner) *Given a cryptographic primitive* P *implementing a set of algorithms indexed by* $j \in [|\mathsf{P}|]$. *A robust* $(n;k)$ *black-box DAG* P*-combiner is a* $|\mathsf{P}|$*-tuple*

$$\left(\mathfrak{G}_j = \left(\mathcal{G}_j, c_j, s_j, t_j, \{f_{j,v}\}_{v \in V(\mathcal{G}_j)}\right)\right)_{j \in [|\mathsf{P}|]}$$

where $\mathcal{G}_j$ *is a directed acyclic graph with vertex set* $V(\mathcal{G}_j)$, $c_j : V(\mathcal{G}_j) \setminus \{s_j, t_s\} \to [n] \times [|\mathsf{P}|]$ *is a vertex-coloring with colors* (a, b) *indicating that the corresponding query is made to algorithm* a *of candidate oracle* b, *source and sink* $s_j, t_s \in V(\mathcal{G}_j)$ *satisfying* $\mathcal{N}^-(s_j) = \mathcal{N}^+(t_j) = \varnothing$ *and for simplicity the only vertices satisfying those properties and finally* $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ *are ppt. Turing machines. satisfying that the following tuple of oracle Turing machines is a robust* $(n;k)$ *black-box combiner.*

$$\widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}} = \left(\widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}}.[\![\mathfrak{G}_1]\!], \ldots, \widetilde{\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}}.[\![\mathfrak{G}_{|\mathsf{P}|}]\!]\right)$$

Where $[\![\mathfrak{G}_i]\!]$ *is the interpretation according to Figure 3.1.*

Algorithm $[\![\mathcal{G}, c, s, t, \{f_v\}_{v\in V(\mathcal{G})}; \rho]\!](\mathbf{x})$

$\mathbf{y}_s \leftarrow \mathbf{x}$
for $v \in \tau(\mathcal{G}, s) \setminus \{s, t\}$ **do**
  $\mathbf{x}_v \leftarrow f_v(\mathbf{x}, \{(\mathbf{y}_w, w); \rho\}_{w\in\mathcal{N}^-(v)})$
  $\mathbf{y}_v \leftarrow \mathcal{O}_{c(v)}(\mathbf{x}_v)$
return $f_t(\mathbf{x}, \{\mathbf{y}_w\}_{w\in\mathcal{N}^-(t)})$

Algorithm $\tau(\mathcal{G}, s)$

$S \leftarrow \{s\}$
$L \leftarrow []$
while $S \neq \varnothing$
  $u \leftarrow \text{lin} \leftarrow \{s\}$
$q \leftarrow \{s\}$
while $q \neq \varnothing$
  $q' \leftarrow \varnothing$
  for $v \in q$
    $\text{lin} \leftarrow \text{lin} \cup \mathcal{N}^+(v)$
    $\mathrm{q}' \leftarrow q' \cup \mathcal{N}^+(v)$
  $q \leftarrow q'$
return lin

Figure 3.1: Interpretation of $\left(\mathcal{G}, c, s, t, \{f_v\}_{v\in V(\mathcal{G})}\right)$ into an algorithm. τ returns the vertices in $\mathcal{G}$ in topological order starting from the source s.

Example. We show the robust $(2;1)$ black-box DAG combiner for verifiable random functions with guaranteed unique provability in Section 3.1 by drawing the graphs corresponding to each algorithm and labling each vertex with the corresponding oracle queried and the explicit description Turing machine modeling the preceeding computation. For clarity we will use the name of the algorithms rather than an indexing and we label the vertices with the corresponding oracle calls direclty.

$$s_{\mathsf{Gen}_{\mathsf{VRF}}}$$

$$\mathcal{O}_1.\mathsf{Gen}_{\mathsf{VRF}} \qquad \mathcal{O}_2.\mathsf{Gen}_{\mathsf{VRF}}$$

$$t_{\mathsf{Gen}_{\mathsf{VRF}}}$$

Figure 3.2: Drawing of the graphs $\mathcal{G}$

We will refer to a robust black-box DAG combiner as a DAG combiner. According to the definition Definition 3.1 every DAG combiner is necessarily also a robust black-box combiner. In the following we show that the converse also holds, that is, that the class of DAG combiners and robust black-box combiners are computationally equivalent for any underlying primitive.

Theorem 3.2 (Generality of Robust $(n;k)$ Black-Box DAG Combiners) *For every robust $(n;k)$ black-box combiner there exists a computationally equivalent*

robust $(n;k)$ *black-box DAG combiner.*

Informally this equality holds as every robust black-box combiner consists of algorithms that performs computations interleaved with oracle queries to their candidate schemes. We can unroll this computation create a node for each oracle query performed during the computation to arrive at the corresponding vertex-colored DAG, the computations performed in-between oracle queries can be represented with a ppt. Turing machine $f_{j,i}$.

Proof (Theorem 3.2) Let P be a cryptographic primitive Π_{P} with defining algorithms $(p_1, \ldots, p_{|\mathsf{P}|})$, et Π_{P} be a robust $(n;k)$ black-box P-combiner according to Definition 2.16, and let $j \in [|\mathsf{P}|]$ be arbitrary. By Definition 2.16 there exists a polynomial $T \in \mathsf{poly}(n, \lambda, |x|)$ where x is the input to p_j. We construct a directed acyclic graph $\mathcal{G}_j$

$$\mathcal{G}_j = ([n] \times [|\mathsf{P}|] \times [T] \cup \{s,t\}, \{\, ((a_1,b_1,t_1),(a_2,b_2,t_2)) : t_1 < t_2 \,\})$$

We define the coloring by $c(a,b,t) = (a,b)$. We define the Turing machines $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ iteratively starting from the beginning of the computation s, we define $f_{j,v_{j,a,b,1}}$ to be the ppt Turing machine that simulates the $\Pi_{\mathsf{P}}.p_j$ until the first call to its oracle. If that oracle happens to be (a,b) then we end, otherwise write 0 onto the tape. In every subsequent level $f_{j,a,b,t+1}$ simulates the combiner on and answers the queries to the respective oracle with the output of that query ($y_{j,a,b,k}$ for $k \in [t]$) by progressively simulating all previous Turing machines $f_{j,a,b,k}$ for $k \in [t]$. By checking whether after the computation we find a 0 on the tape we can trace the path of queried oracles. We can now continue the simulation of the combiner since we know exactly the state of the tape. The combiner is necessarily efficient since for every the number of vertices in the graph is bounded from above by $nT(n,\lambda,|x|)|\mathsf{P}|$ where $T \in \mathsf{poly}(m,\lambda,|x|)$, we only make exactly one query to any oracle per vertex (which also puts a polynomial bound on the number of oracle calls), the topological ordering can be computed in polynomial time. We are only left to show that the Turing machines $\{f_{j,v}\}_{v \in V(\mathcal{G}_j)}$ are indeed ppt. To this end note that $f_{j,v}$ simulates the black-box combiner Turing machine, which is assumed to be ppt, at most once where it can answer any oracle query by looking up the corresponding output of the layer in time polynomial in $n, |\mathsf{P}|$ and λ. □

3.2 Impossibility of Robust Black-Box VRF-Combiner

We consider the possibility of combining VRFs in a black-box fashion. As previously mentioned we present an impossibility result for general robust $(n; n-1)$ black-box VRF-combiners. This means that given n VF oracles there is no way to combine them in a construction such that the construction is VRF secure if at least $n-1$ candidates are secure. For simplicity we assume that the secret key spaces of the candidates are of the form $\mathcal{SK} = \{0,1\}^{s(\lambda)}$ for some $s \in \mathsf{poly}(\lambda)$.

Theorem 3.3 *There exists no robust* $(n; n-1)$ *black-box* VRF*-combiner.*

Intuition. We need to show that for every $(n; n-1)$ black-box VRF-combiner Π_{VRF} there exists a tuple $\mathbf{O} = (\mathcal{O}_1, \ldots, \mathcal{O}_n)$ of VF oracles of which at least $n-1$ implement a secure VRF, the combiner $\Pi^{\mathbf{O}}_{\mathsf{VRF}}$ doesn't implement a secure VRF. By equivalence of robust black-box combiner and robust black-box DAG combiners in Theorem 3.2 we instead show the statement for robust black-box DAG combiners. For that we fix an arbitrary robust black-box DAG combiner Π_{VRF} with the DAGs $G_{gen}, G_{eval}, G_{vfy}$ with sources $s_{gen}, s_{eval}, s_{vfy}$, sinks $t_{gen}, t_{eval}, t_{vfy}$, colorings $c_{gen}, c_{eval}, c_{vfy}$ and ppt. algorithms $\{f_v^{gen}\}_{v \in V(G_{gen})}$, $\{f_v^{eval}\}_{v \in V(G_{eval})}$ and $\{f_v^{vfy}\}_{v \in V(G_{vfy})}$ for the algorithms $\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}$ and $\mathsf{Vfy}_{\mathsf{VRF}}$ respectively. We fix a topological ordering of vertices $v_1, \ldots, v_q$ in G_{eval} and iterate over them. Since we assume verification and evaluation algorithms to be deterministic the the randomness from queries to the key-generation algorithm cannot influence the evaluation or verification output, therefore those queries can replaced with a hard-coded key. Additionally since all oracle queries to eval and verification are determiniistic we assume that for every vertex the query input for each oracle is unique (since we can just connect an edge to the first query, in topolofical ordering, and use this instead of requerying). We finally make the assumption that $\mathcal{SK} = \{0,1\}^{s(\lambda)}$ for $s \in \mathsf{poly}(\lambda)$.

Let $v_1, \ldots, v_n$ be a topological ordering of the vertices in G_{eval}. We iterate over the vertices in the order $v_1, \ldots, v_n$ and distinguish the following cases: Either for all uniquely provable and perfectly correct candidate oracles $\mathcal{O}_1, \ldots, \mathcal{O}_n$ and secret keys sk, and evaluation points x, the output of the query made corresponding to v_i does not influence the output at all, that is if we change the output of the query associated to v_1 the output y of the combiner remains the same. We call such vertices *irrelevant*. Irrelevant vertices can be simulated by an adversary by using any arbitrary value.

Since we know that all prior vertices are irrelevant we know that the function associated with the vertex v_i, $f_{v_i}^{eval}(\mathsf{sk}, x, o_1, \ldots, o_{d^+(v_i)})$, can be replaced by a function $\tilde{f}_{v_i}^{eval}(\mathsf{sk}, x) = f_{v_i}^{eval}(\mathsf{sk}, x, r_1, \ldots, r_{d^+(v_i)})$ where the values of r are obtained by arbitrary values in the support of the corresponding oracles.

Assume for all uniquely provable and perfectly correct candidates $\mathbf{O}$ we have that replacing the true sk in the execution of the evaluation algorithm with another $\widehat{\mathsf{sk}}, \widetilde{\mathsf{sk}} \leftarrow_{\$} \Pi_{\mathsf{VRF}}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$ the output of the combiner doesnt change, except with negligible probability, then this query is also simulatable as the adversary can just query the corresponding oracle on (sk', x) where $\mathsf{sk}' \leftarrow_{\$} \Pi^{\mathbf{O}}_{\mathsf{VRF}}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$ and with overwhelming probability the oracle produces the same output. We call those queries *public*. Public queries can also easily be simulated.

If we arrive at a non-irrelevant and non-public vertex v_j then we first claim that the verification algorihm necessarily needs to requery the exact same query. If that were not the case then we can construct VRF secure candidates (except for the oracle corresponding to this non-irrelevant and non-public vertex), and only let this oracle differ for that exact evaluation. Since at least $n-1$ candidates are VRF secure, the combiner also needs to be VRF secure, in particular uniquely provable and perfectly correct. By perfect correctness the two output however $\hat{y} \neq \tilde{y}$ both need to verify in their respective world. On the other hand in the other world they can not verify unless they break unique provability of the combiner. Since the two worlds can only be distinguished by this single oracle evaluation we conclude that the verification algorithm must rerun this query.

Let $T \subseteq V(G_{eval})$ be the set of vertices that are not irrelevant and not public. We now construct the following pseudorandomness adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$. $\mathcal{A}_1$ plays the experimentation phase of the prf security experiment, it computes $2s(\lambda)$ many input-evaluation pairs $\{(x_i, y_i, \pi_i)\}_{i \in 2s(\lambda)}$. Now it iterates over all evaluation pairs $i \in [2s(\lambda)]$ and iteratively filters the set of secret keys $\mathcal{SK}$ using its PSPACE oracle PSPACE. The filtering works as follows: For every secret key candidate $\mathsf{sk} \in \mathcal{SK}'$ simulate the computation of the evaluation algorithm, irrelevant and public vertices are easily simulated. For vertices in T follow the evaluation graph and once we would make a query we check whether the query q is in the transcript $\mathcal{T}_i$, if the immediate query doesn't occur in $\mathcal{T}_i$ the secret key sk can be thrown out. Otherwise if it occurs in $\mathcal{T}_i$ then we continue with that oracle response. If the final output y differs from the obtained y_i then we can also toss it outside, otherwise the secret key sk remains a valid candidate.

Note that due to correctness we never discard the correct secret key $\mathsf{sk}^* \in \mathcal{SK}$, if in every second filtration we drop at most half the secret keys from the previous candidates then with high probability, on a uniformly random value $x \leftarrow_{\$} \{0,1\}^*$ we win the pesudorandomness game. Otherwise if at least half of the previous candidates are discarded we end up with a polynomially bounded set of remaining secret keys $\mathcal{SK}^*$ from which we can guess the correct secret key with significant success probability and also win the pseudorandomness game.

Proof (Theorem 3.3) Fix a robust $(n; n-1)$ black-box DAG VRF-combiner Π_{VRF} and consider the directed acyclic graph $\mathcal{G}_{eval}$ corresponding to the evaluation algorithm (see Definition 3.1). First assume that every vertex $v \in V(\mathcal{G}_{eval})$ is *ignored* in the following sense:

Definition 3.4 (Ignored vertices for $\mathcal{G}_{eval}$)

Claim 3.5 *If every vertex $v \in V(\mathcal{G}_{eval})$ is ignored according to Definition 3.4 then there exists an efficient (ppt) adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ which breaks psueudorandomness of the combiner.*

Since we arrive at a contradiction the converse of the initial assumption must hold, that is that not every vertex $v \in V(\mathcal{G}_{eval})$ is ignored. Then there exists a secret key $\mathsf{sk} \in \mathcal{SK}$, evaluation point $x \in \{0,1\}^\lambda$, oracles $\mathbf{O} = (\mathcal{O}_1, \ldots, \mathcal{O}_n)$ and values $\hat{z}, \tilde{z}$ such that

$$\hat{y} = \Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\ldots,\mathcal{O}_i[v\mapsto\hat{z}],\ldots,\mathcal{O}_n}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x) \neq \Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\ldots,\mathcal{O}_i[v\mapsto\tilde{z}],\ldots,\mathcal{O}_n}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x) = \tilde{y}$$

Where $\mathcal{O}_i$ is the candidate oracle associated with v. We now construct an oracle $\mathbf{O}^* = (\mathcal{O}_1^*, \ldots, \mathcal{O}_i, \ldots, \mathcal{O}^*)$

TODO

Suppose, towards a contradiction that there exists a robust $(n; n-1)$ black-box VRF-combiner Π_{VRF} according to **??**. By Theorem 3.2 there also exists a robust black-box DAG combiner defined in Definition 3.1 defined by

$$\left(\mathcal{G}_j, c_j, s_j, t_j, \{f_{j,v}\}_{v\in V(\mathcal{G}_j)}\right)_{j\in[3]}$$

We now consider $\mathcal{G}_2$ which corresponds to the DAG of the combiners evaluation algorithm.

Adversary $\mathcal{A}_1^{\mathsf{PSPACE},\mathcal{O}^{prf}_{\Pi^{\mathsf{O}}_{\mathsf{VRF}}}}(\mathsf{pk})$

$\mathcal{SK}' \leftarrow \mathcal{SK}$
for $i \in [2s(\lambda)]$ **do**
  $x_i \leftarrow_{\$} \{0,1\}^{\lambda}$
  $(y_i, \pi_i) \leftarrow_{\$} \mathcal{O}^{prf}_{\Pi^{\mathsf{O}}_{\mathsf{VRF}}}(x_i)$
  $\mathcal{T}_i \leftarrow \mathsf{transcript}(\Pi^{\mathsf{O}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x_1, y_1, \pi_1))$
  // Precompute responses for irrelevant and public vertices:
  $\widetilde{\mathsf{sk}} \leftarrow_{\$} \mathcal{SK}$
  $f_{sim} : v_i \mapsto \mathcal{O}_{c(v_i)}(\tilde{f}_{v_i}(\mathsf{sk}, i))$
  // Filter the secret keys:
  $\mathcal{SK}' \leftarrow \{\mathsf{sk} \in \mathcal{SK} : \mathcal{S}^{\mathsf{PSPACE}}(\mathsf{sk}, f_{sim}, x_i, y_i, \pi_i, \mathcal{T}_i) = 1\}$
$\mathsf{sk}^* \leftarrow_{\$} \mathcal{SK}'$
$\sigma \leftarrow \mathsf{sk}^*$
return σ

Adversary $\mathcal{A}_2^{\mathcal{O}^{prf}_{\Pi^{\mathsf{O}}_{\mathsf{VRF}}}}(\mathsf{pk}, x, y, \sigma)$

$\mathsf{sk}^* \leftarrow \sigma$
$y^* \leftarrow \Pi^{\mathsf{O}}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}^*, x)$
return $y^* = y$

Simulator $\mathcal{S}^{\mathsf{PSPACE}}(\mathsf{sk}, f^{eval}_{sim}, x, y, \pi, \mathcal{T})$

// All of this is computed in $\mathcal{O}(1)$ due to PSPACE complete oracle.
$v_1, \ldots, v_{|V(G_{eval})|} \leftarrow \tau(V(G_{eval}))$
for $i \in [|V(G_{eval})|]$ **do**
  if $v_i \in \mathcal{V}_{irrelevant} \cup \mathcal{V}_{public}$ **then**
    $w_i \leftarrow f^{eval}_{sim}(v_i)$
  else
    $u_i \leftarrow f^{eval}_{v_i}(\mathsf{sk}, x, \{(w_i, v_i) : v_i \in \mathcal{N}^{+}_{G_eval}(v)\})$
    if $u_i \notin \mathcal{T}$ at an oracle query to $\mathcal{O}_{c(v)}$ **then**
      return 0 // if the query is not repeated in verify alg.
    else
      $w_i \leftarrow \mathcal{T}(u_i)$ // use the response of the oracle in verify alg.
return $w_t = (y, \pi)$

Figure 3.3: Combiner for VRFs if all candidate schemes are uniquely provable.

Using the fact that both VRF and VUF satisfy strictly stronger notions of security than wVRF do we conclude that if we cannot combine VRFs to obtain a secure wVRF then we can also not combine VRFs to obtain a VRF or VUF. The two following corollary formulate implications of Theorem 3.3.

Corollary 3.6 (Impossibility of a Robust $(n; n-1)$ Black-Box VRF-combiner) *Given n secure VRF candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$ there exists no robust black-box combiner that implements a secure VRF scheme if even one candidate scheme is not uniquely provable.*

Corollary 3.7 (Impossibility of a Robust $(n; n-1)$ Black-Box VUF-combiner) *Given n secure VUF candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$ there exists no robust black-box combiner that implements a secure VUF scheme if even one candidate scheme is not uniquely provable.*

As we have now ruled out the possibility of combining VRFs in a black-box fashion we can either consider even stronger relaxations of the VRFs security properties or we can strengthen our assumptions and consider more advanced primitives. Because unique provability is an information theoretic property that cannot rely on any computational hardness assumptions and is mostly mathematical proven for good constructions it is plausible to assume all candidates to satisfy unique provability. Many candidates schemes that implement secure VRFs can indeed be combined if we additionally assume that every candidate scheme is uniquely provable. The combiner is given by a slightly modified combiner for pseudorandom functions. A robust $(2;1)$ black-box combiner is outlined in Figure 3.4, note that from that we can construct a robust $(n;1)$ black-box combiner by simply chaining the combiners. We note that this combiner construction, with minor extensions can constrainable VRFs for the generic constrain algorithm, and key-homomorphic VRFs if we additionally publicly verify a statistical notion of key-homomorphism using an efficient homomorphism check by Blum ...
We now consider additional security properties for VRFs, aggregatability that allows the aggregation of many secret keys or image-proof pairs to allow for efficient batch verification and unbiasability which prevents adversaries from skewing the output distribution of a VRF by maliciously selecting the keys.

Algorithm $\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$

1 : $(\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow_\$ \mathcal{O}_1.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
2 : $(\mathsf{sk}_2, \mathsf{pk}_2) \leftarrow_\$ \mathcal{O}_2.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
3 : **return** $((\mathsf{sk}_1, \mathsf{sk}_2), (\mathsf{pk}_1, \mathsf{pk}_2))$

Algorithm $\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$

1 : $(\mathsf{sk}_1, \mathsf{sk}_2) \leftarrow \mathsf{sk}$
2 : $(y_1, \pi_1) \leftarrow \mathcal{O}_1.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, x)$
3 : $(y_2, \pi_2) \leftarrow \mathcal{O}_2.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_2, x)$
4 : $y \leftarrow y_1 \oplus y_2$
5 : $\pi \leftarrow (y_1, \pi_1, y_2, \pi_2)$
6 : **return** (y, π)

Algorithm $\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$

1 : $(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$
2 : $(y_1, \pi_1, y_2, \pi_2) \leftarrow \pi$
3 : **req** $y = y_1 \oplus y_2$
4 : **req** $\mathcal{O}_1.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_1, x, y_1, \pi_1) = 1$
5 : **req** $\mathcal{O}_2.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_2, x, y_2, \pi_2) = 1$

Figure 3.4: Combiner for VRFs if all candidate schemes are uniquely provable.

Robust Black-Box Combiner for Unbiasable VRF. We present a robust black-box combiner for unbiasable VRF under the assumption that all candidate schemes are uniquely provable. We first note that there exists a compiler to reduce unbiasable VRF to VRF under computational Diffie-Hellman (CDH) assumption [**?**], a combiner that works under the CDH assumption would be to ignore unbiasability of the candidates and combining normal VRFs and then introducing unbiasability afterwards . Instead of relying on an additional hardness assumption we prefer to use the unbiasability property of the secure candidates. We introduce a robust $(2;1)$ black-box combiner for unbiasable VRF based on collision resistant hash functions and randomness extractors that is heavily inspired by the construction of unbiasable VRF presented in [**?**]. The construction works by initially splitting the input $x \in \{0,1\}^\lambda$ into two parts $x = x_0 \| x_1$ then hashing $x_1 \| 0$ and $x_1 \| 1$ using a collision resistant hash function. Finally we evaluate each of the candidate on one point $H(x_1 \| 0)$ and $H(x_1 \| 1)$ before running a randomness extractor to extract randomness from x_0 and adding it to $y_0 \oplus y_1$. We formulate the result formally as a theorem.

is this even a robust black-box combiner since we cannot reduce unbiasability from the combiner to its candidates.

Theorem 3.8 (Robust (2;1) Black-Box Combiner for Unbiasabile VRF) *There exists a uniform robust $(2;1)$ black-box combiner for unbiasable VRF if all candidate schemes are uniquely provable.*

Robust Black-Box Combiner for Aggregatable VRF. We first look at key-homomorphism combiners. Given a key-homomorphic function given an evaluation point $x \in \{0,1\}^\lambda$ we show how to publicly check for the homomorphism property of each candidate scheme. This is possible due to an algorithm by Blum [?] which introduces an efficient homomorphism check that runs in time independent of the group size. This means that given an evaluation point we can check whether the evaluation function is a group homomorphism with respect to the secret-key argument with high probability in time independent of the size of the secret-key space. We state this claim in form of a lemma and quickly elaborate on the homomorphism check, referring to the original result for the complete proof.

Lemma 3.9 (Robust (2;1) Black-Box Combiner for Key-homomorphic VRF)

Next we consider aggregable VRFs. We show that indeed there exists a combiner

3.3 Discussion and Extensions

There are different ways to combine VRFs, for both unique provability and key-homomorphism they cannot be combined in a black-box fashion if we want to preserve properties such as unpredictability and therefore also randomness. On the other hand we manage to construct a robust black-box combiner that combines unbiasabile VRFs under guaranteed unique provability for all candidates. We find that the black-box combination of information theoretic properties generally seems hard if other properties such as unpredictability need to be preserved. While the class of all robust black-box combiners initially seems to be very broad we show that we can capture all polynomially bounded combiners using the notion of a robust black-box DAG combiner, which allows a more systematic analysis of possible constructions by abstracting a graph structure in the computation executed by the implemented algorithms. We have not considered possible combiner constructions for stronger relaxations, such as computational unique provability or correctness with error, this is mainly because the resulting construction only offer weak guarantees that are insufficient for practical applications.

Chapter 4

Conclusion

Appendix

Proof of ??

Proof It is sufficient to find a $(1;2)$-robust black-box combiner for VRF where all candidates are uniquely provable. Given a combiner we can then construct a $(1;n)$-robust black-box combiner for VRF by nesting the combiners iteratively. A construction for the combiner is given in **??**. The key generation works by querying both candidates on the given security parameter and grouping the secret and public keys respectively. Evaluation again queries both candidate schemes on the input with their respective secret key and returns the xor of the outputs while the the individual outputs and proofs consistute the new proof. The verification algorithm checks whether the image y is the result of xoring the individual outputs of the candidates and whether the individual images y_1, y_2 verify with respect to the verification key the proof and the pre-image. To ensure that the combiner is still efficient we sim

The combiner is clearly efficient since every algorithm consists of one call to the respective algorithm of each candidate scheme (whi)

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).